

Library Management System

Simple Dataset + UML Diagrams

5 Tables • UML Class Diagram • ER Crow's Foot • Sample Data • SQL

Members • Books • Categories • Authors • Loans

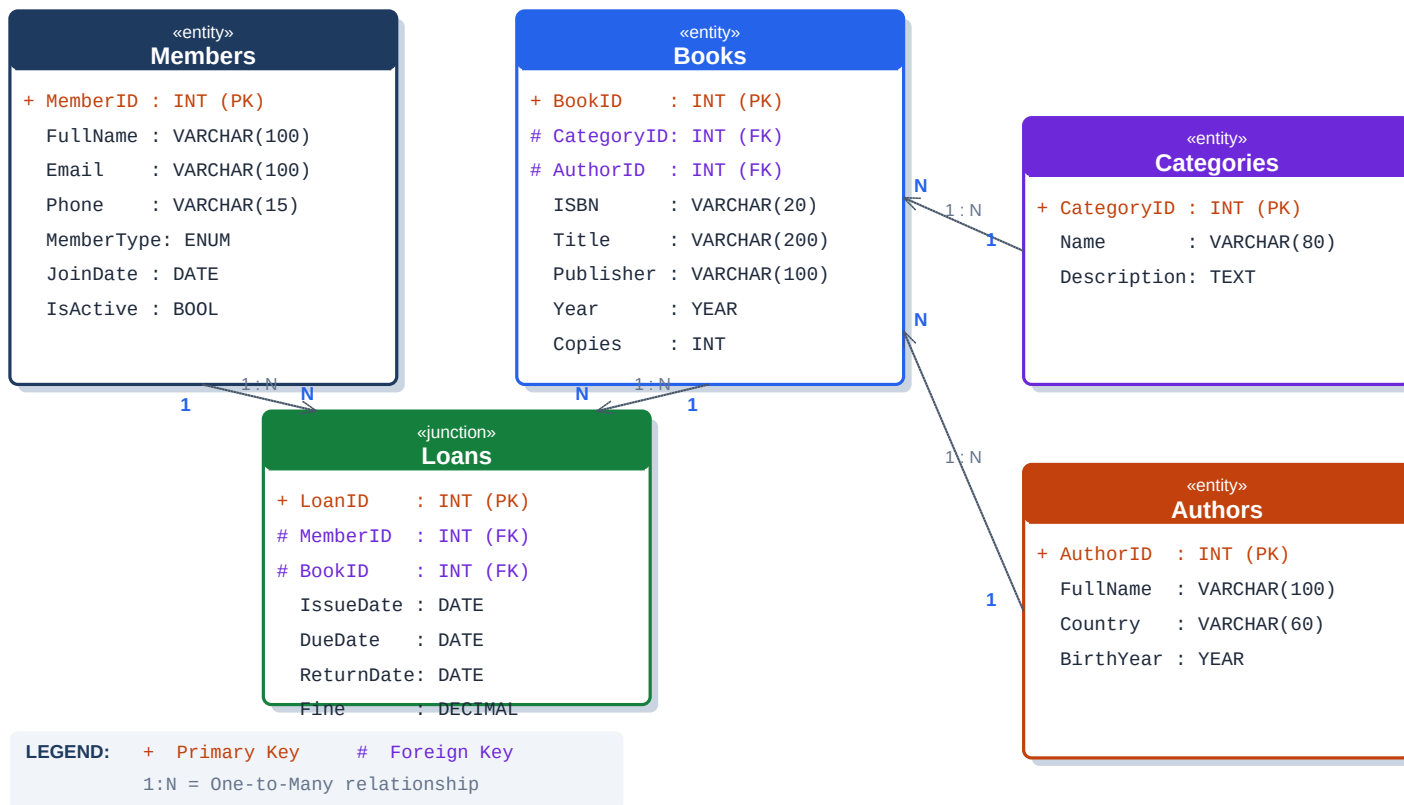
This guide shows a simple **Library Management System** — 5 tables, realistic sample data, a full **UML Class Diagram**, a **Crow's Foot ER Diagram**, and the complete SQL to build it.

Tables at a Glance

Table	Rows	Primary Key	Purpose
Members	8	MemberID (AUTO)	Library card holders
Categories	5	CategoryID (AUTO)	Book genres/subjects
Authors	6	AuthorID (AUTO)	Book authors
Books	10	BookID (AUTO)	Book catalogue
Loans	12	LoanID (AUTO)	Borrow/return records

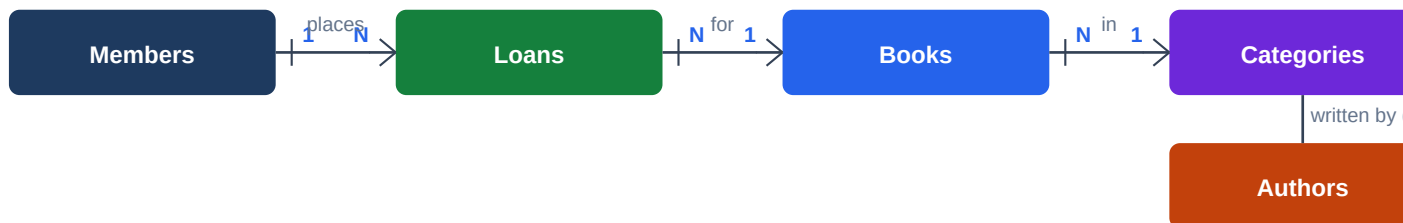
UML Class Diagram

The UML Class Diagram shows each table as a class with its attributes and data types. + = Primary Key (orange) • # = Foreign Key (purple) • Lines show relationships with multiplicity.



Crow's Foot ER Diagram

Crow's Foot notation shows cardinality — the **single vertical bar** means **one** and the **three-pronged fork** means **many**.



Relationships Explained

Relationship	Type	Rule	Example
Categories → Books	1 : N	One category has many books	Fiction has 4 books
Authors → Books	1 : N	One author writes many books	Yuval Harari wrote Sapiens & Homo Deus
Members → Loans	1 : N	One member can borrow many books	Arjun has 3 active loans
Books → Loans	1 : N	One book can be loaned many times	'The Alchemist' loaned 3 times total

Relationship	Type	Rule	Example
Members & Books via Loans	M: N	Many members borrow many books	Implemented through the Loans table

SQL — Create All Tables

```
CREATE DATABASE LibraryDB; USE LibraryDB; CREATE TABLE Categories ( CategoryID INT PRIMARY KEY AUTO_INCREMENT, Name VARCHAR(80) NOT NULL UNIQUE, Description TEXT ); CREATE TABLE Authors ( AuthorID INT PRIMARY KEY AUTO_INCREMENT, FullName VARCHAR(100) NOT NULL, Country VARCHAR(60), BirthYear YEAR ); CREATE TABLE Books ( BookID INT PRIMARY KEY AUTO_INCREMENT, CategoryID INT NOT NULL, AuthorID INT, ISBN VARCHAR(20) UNIQUE NOT NULL, Title VARCHAR(200) NOT NULL, Publisher VARCHAR(100), Year YEAR, Copies INT DEFAULT 1 CHECK(Copies >= 0), FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID) ON DELETE RESTRICT, FOREIGN KEY (AuthorID) REFERENCES Authors(AuthorID) ON DELETE SET NULL ); CREATE TABLE Members ( MemberID INT PRIMARY KEY AUTO_INCREMENT, FullName VARCHAR(100) NOT NULL, Email VARCHAR(100) UNIQUE NOT NULL, Phone VARCHAR(15), MemberType ENUM('Student','Faculty','Public') DEFAULT 'Public', JoinDate DATE DEFAULT (CURRENT_DATE), IsActive BOOLEAN DEFAULT TRUE ); CREATE TABLE Loans ( LoanID INT PRIMARY KEY AUTO_INCREMENT, MemberID INT NOT NULL, BookID INT NOT NULL, IssueDate DATE DEFAULT (CURRENT_DATE), DueDate DATE NOT NULL, ReturnDate DATE, Fine DECIMAL(6,2) DEFAULT 0.00 CHECK(Fine >= 0), Status ENUM('Active','Returned','Overdue') DEFAULT 'Active', CONSTRAINT chk_due CHECK(DueDate > IssueDate), FOREIGN KEY (MemberID) REFERENCES Members(MemberID) ON DELETE RESTRICT, FOREIGN KEY (BookID) REFERENCES Books(BookID) ON DELETE RESTRICT );
```

Sample Dataset

Categories

CategoryID	Name	Description
1	Fiction	Novels, short stories and literary works
2	Non-Fiction	Biographies, history, science and essays
3	Science	Physics, biology, chemistry, mathematics
4	Technology	Programming, AI, databases, engineering
5	Philosophy	Ethics, logic, existentialism, metaphysics

Authors

AuthorID	FullName	Country	BirthYear
1	Harper Lee	USA	1926
2	Yuval Noah Harari	Israel	1976
3	Stephen Hawking	UK	1942
4	Paulo Coelho	Brazil	1947
5	Robert C. Martin	USA	1952
6	Fyodor Dostoevsky	Russia	1821

Books

BookID	Title	AuthorID	CategoryID	Year	Copies
1	To Kill a Mockingbird	1	1	1960	3

2	Sapiens	2	2	2011	4
3	A Brief History of Time	3	3	1988	2
4	The Alchemist	4	1	1988	5
5	Clean Code	5	4	2008	3
6	Crime and Punishment	6	1	1866	2
7	Homo Deus	2	2	2015	3
8	The Grand Design	3	3	2010	2
9	The Pragmatic Programmer	5	4	1999	4
10	The Brothers Karamazov	6	5	1880	1

Members

MemberID	FullName	Email	MemberType	JoinDate
1	Arjun Sharma	arjun.s@email.com	Student	2023-06-01
2	Priya Nair	priya.n@email.com	Student	2023-06-15
3	Rahul Verma	rahul.v@email.com	Faculty	2022-01-10
4	Sneha Rao	sneha.r@email.com	Student	2024-01-05
5	Karthik Pillai	karthik.p@email.com	Public	2023-09-20
6	Divya Menon	divya.m@email.com	Student	2024-02-14
7	Arun Kumar	arun.k@email.com	Faculty	2021-08-01
8	Lakshmi Devi	lakshmi.d@email.com	Public	2023-11-30

Loans

LoanID	MemberID	BookID	IssueDate	DueDate	ReturnDate	Fine	Status
1	1	4	2024-06-01	2024-06-15	2024-06-14	0.0	Returned
2	1	5	2024-06-10	2024-06-24		0.0	Active
3	2	2	2024-05-20	2024-06-03	2024-06-10	35.0	Returned
4	2	7	2024-06-12	2024-06-26		0.0	Active
5	3	1	2024-06-01	2024-07-01		0.0	Active
6	3	3	2024-05-01	2024-05-15	2024-05-14	0.0	Returned
7	4	9	2024-06-05	2024-06-19		0.0	Active
8	5	4	2024-05-15	2024-05-29	2024-06-02	40.0	Returned
9	6	6	2024-06-08	2024-06-22		0.0	Active
10	7	2	2024-04-01	2024-04-15	2024-04-14	0.0	Returned
11	7	8	2024-06-01	2024-07-01		0.0	Active
12	8	10	2024-06-10	2024-06-24		0.0	Active

Sample Queries

Q1 — All active loans with member and book name:

```
SELECT m.FullName, b.Title, l.IssueDate, l.DueDate, l.Status FROM Loans l JOIN Members m ON  
m.MemberID = l.MemberID JOIN Books b ON b.BookID = l.BookID WHERE l.Status = 'Active' ORDER  
BY l.DueDate;
```

Q2 — Books with total times borrowed:

```
SELECT b.Title, a.FullName AS Author, COUNT(l.LoanID) AS TimesBorrowed FROM Books b LEFT  
JOIN Authors a ON a.AuthorID = b.AuthorID LEFT JOIN Loans l ON l.BookID = b.BookID GROUP BY  
b.BookID ORDER BY TimesBorrowed DESC;
```

Q3 — Members with overdue books (return date passed):

```
SELECT m.FullName, m.Email, b.Title, l.DueDate, DATEDIFF(CURRENT_DATE, l.DueDate) AS  
DaysOverdue FROM Loans l JOIN Members m ON m.MemberID = l.MemberID JOIN Books b ON b.BookID =  
l.BookID WHERE l.Status = 'Active' AND l.DueDate < CURRENT_DATE;
```

Q4 — Total fines collected per member:

```
SELECT m.FullName, SUM(l.Fine) AS TotalFine FROM Members m JOIN Loans l ON l.MemberID =  
m.MemberID WHERE l.Fine > 0 GROUP BY m.MemberID ORDER BY TotalFine DESC;
```